Phase 3 Mock Code Challenge: Freebie Tracker

- Due No Due Date
- Points 1
- Submitting a website url

FORK (https://github.com/learn-co-curriculum/python-p3-freebie-tracker/fork) (https://github.com/learn-co-curriculum/python-p3-freebie-tracker) (https://github.com/learn-co-curriculum/python-p3-freebie-tracker/issues/new)

Learning Goals

- Write SQLAlchemy migrations.
- Connect between tables using SQLAlchemy relationships.
- Use SQLAlchemy to run CRUD statements in the database.

Key Vocab

- **Schema**: the blueprint of a database. Describes how data relates to other data in tables, columns, and relationships between them.
- **Persist**: save a schema in a database.
- **Engine**: a Python object that translates SQL to Python and vice-versa.
- **Session**: a Python object that uses an engine to allow us to programmatically interact with a database.
- **Transaction**: a strategy for executing database statements such that the group succeeds or fails as a unit.
- **Migration**: the process of moving data from one or more databases to one or more target databases.

Introduction

For this assignment, we'll be working with a freebie domain.

As developers, when you attend hackathons, you'll realize they hand out a lot of free items (informally called *freebies*, or swag)! Let's make an app for developers that keeps track of all the freebies they obtain.

We have three models: `Company`, `Dev`, and `Freebie`

For our purposes, a `Company` has many `Freebie`s, a `Dev` has many `Freebie`s, and a `Freebie` belongs to a `Dev` and to a `Company`.

`Company` - `Dev` is a many to many relationship.

Note: You should draw your domain on paper or on a whiteboard *before you start coding*. Remember to identify a single source of truth for your data.

Instructions

To get started, run `pipenv install && pipenv shell` while inside of this directory.

Build out all of the methods listed in the deliverables. The methods are listed in a suggested order, but you can feel free to tackle the ones you think are easiest. Be careful: some of the later methods rely on earlier ones.

Remember! This mock code challenge does not have tests. You cannot run `pytest` and you cannot run `learn test`. You'll need to create your own sample instances so that you can try out your code on your own. Make sure your relationships and methods work in the console before submitting.

We've provided you with a tool that you can use to test your code. To use it, run `python debug.py` from the command line. This will start an `ipdb` session with your classes defined. You can test out the methods that you write here. You are also encouraged to use the `seed.py` file to create sample data to test your models and associations.

Writing error-free code is more important than completing all of the deliverables listed- prioritize writing methods that work over writing more methods that don't work. You should test your code in the console as you write.

Similarly, messy code that works is better than clean code that doesn't. First, prioritize getting things working. Then, if there is time at the end, refactor your code to adhere to best practices.

Before you submit! Save and run your code to verify that it works as you expect. If you have any methods that are not working yet, feel free to leave comments describing your progress.

What You Already Have

The starter code has migrations and models for the initial `Company` and `Dev` models, and seed data for some `Company` s and `Dev` s. The schema currently looks like this:

companies Table

Column	Type
name	String
founding_year	Integer

devs Table

Column	Type
name	String

You will need to create the migration for the `freebies` table using the attributes specified in the deliverables below.

Deliverables

Write the following methods in the classes in the files provided. Feel free to build out any helper methods if needed.

Remember: SQLAlchemy gives your classes access to a lot of methods already! Keep in mind what methods SQLAlchemy gives you access to on each of your classes when you're approaching the deliverables below.

Migrations

Before working on the rest of the deliverables, you will need to create a migration for the `freebies` table.

- A `Freebie` belongs to a `Dev`, and a `Freebie` also belongs to a `Company`. In your migration, create any columns your `freebies` table will need to establish these relationships using the right foreign keys.
- The `freebies` table should also have:
  - An `item_name` column that stores a string.
  - A `value` column that stores an integer.

After creating the `freebies` table using a migration, use the `seed.py` file to create instances of your `Freebie` class so you can test your code.

After you've set up your `freebies` table, work on building out the following deliverables.

Relationship Attributes and Methods

Use SQLAlchemy's `ForeignKey`, `relationship()`, and `backref()` objects to build relationships between your three models.

Note: The plural of "freebie" is "freebies" and the singular of "freebies" is "freebie".

Freebie

- `Freebie.dev` returns the `Dev` instance for this Freebie.
- `Freebie.company` returns the `Company` instance for this Freebie.

Company

- `Company.freebies` returns a collection of all the freebies for the Company.
- `Company.devs` returns a collection of all the devs who collected freebies from the company.

Dev

- `Dev.freebies` returns a collection of all the freebies that the Dev has collected.
- `Dev.companies` returns a collection of all the companies that the Dev has collected freebies from.

Use `python debug.py` and check that these methods work before proceeding. For example, you should be able to retrieve a dev from the database by its attributes and view their companies with `dev.companies` (based on your seed data).

Aggregate Methods

Freebie

- `Freebie.print_details()` should return a string formatted as follows: `{dev name} owns a {freebie item_name} from {company name}` .

Company

- `Company.give_freebie(dev, item_name, value)` takes a `dev` (an instance of the `Dev` class), an `item_name` (string), and a `value` as arguments, and creates a new `Freebie` instance associated with this company and the given dev.
- Class method `Company.oldest_company()` returns the `Company` instance with the earliest founding year.

Dev

- `Dev.received_one(item_name)` accepts an `item_name` (string) and returns `True` if any of the freebies associated with the dev has that `item_name` , otherwise returns `False` .
- `Dev.give_away(dev, freebie)` accepts a `Dev` instance and a `Freebie` instance, changes the freebie's dev to be the given dev; your code should only make the change if the freebie belongs to the dev who's giving it away